

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

- 1. Identify any incorrect or invalid fields in the header.**

The given HTTP response header is:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

The response contains one major error.

Invalid Field:

- **Content-Length: -150**

The **Content-Length** header specifies the size of the response body in bytes. According to the HTTP standard, this value must always be a **non-negative integer**. A negative value such as **-150** is invalid and violates the HTTP protocol.

The remaining fields are valid:

- **HTTP/1.1** specifies the HTTP version.
- **200 OK** indicates that the request was successful.
- **Date** specifies when the response was generated.
- **Content-Type: text/html** tells the browser that the response contains an HTML document.
- **Connection: keep-alive** allows the browser and server to reuse the same TCP connection for multiple requests.

Thus, the primary error in the header is the invalid **Content-Length** value.

2. Analyze the issue with Content-Length and explain its impact.

The **Content-Length** header informs the browser about the exact size of the response body.

Example:

Content-Length: 150

means the browser should receive exactly **150 bytes** of data.

In the given response,

Content-Length: -150

is invalid because the length of a file or webpage cannot be negative.

Impact

- The browser cannot determine the correct size of the webpage.
 - Some browsers may reject the response.
 - The webpage may load incompletely.
 - Images, CSS files, or JavaScript files may fail to load.
 - Browsers may display network or protocol errors.
 - Proxy servers and caches may not store the response correctly.
- Therefore, an incorrect **Content-Length** affects the reliability of communication between the browser and the server.

3. Determine whether the status code matches the response correctness.

The status code is:

200 OK

A **200 OK** status code means that the request was processed successfully and the server is returning a valid response.

However, although the status code indicates success, the response header contains an invalid **Content-Length** value.

Therefore, there is a mismatch between the status code and the actual response.

If the server generates an invalid response header, it should either:

- Correct the header and return **200 OK**, or
- Return an appropriate error code if the response cannot be generated correctly.

Examples include:

- **400 Bad Request**
- **500 Internal Server Error**

Hence, the current response is inconsistent because it reports success while containing an invalid header.

4. Suggest corrections for the identified errors.

The corrected HTTP response header should be:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Additional Recommendations

- Ensure the **Content-Length** matches the actual size of the response.
- Validate response headers before sending them.
- Use HTTP libraries that automatically calculate the content length.
- Test responses using browser developer tools.
- Follow HTTP/1.1 standards for all server responses.

Conclusion

A valid HTTP response header ensures proper communication between the browser and the server. Correct header values improve webpage loading speed, browser compatibility, and overall reliability.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

The given DOM structure is:

HTML

<html>

<head>

<title>TestPage</title>

</head>

<body>

<div>

<p>Hello World

</div>

</body>

</html>

The HTML contains several errors.

Missing Elements

- Missing <!DOCTYPE html>.
- <p> tag is not closed using </p>.
- Semantic HTML5 elements such as <header>, <main>, and <section> are not used.
- The content is placed inside a generic <div> instead of meaningful semantic tags. Properly closed HTML elements improve readability, browser compatibility, and accessibility.

2. Analyze how the browser will interpret this structure.

Modern browsers try to repair invalid HTML automatically.

When the browser reads the above code, it may automatically insert the missing closing </p> tag before the closing </div>.

Different browsers may handle this differently because they use different rendering engines.

Possible browser behavior includes:

- Automatically closing missing tags.
- Rearranging the DOM tree.
- Rendering text differently.
- Applying CSS differently.
- Producing inconsistent layouts.

Although the page may still appear to work, relying on automatic correction is poor programming practice.

3. Determine the impact on rendering and layout.

Invalid HTML affects webpage rendering in several ways.

Possible Effects

- Inconsistent display across browsers.
- Unexpected spacing and alignment.
- CSS rules may not apply correctly.
- JavaScript DOM manipulation may fail.
- Screen readers may interpret the page incorrectly.
- Search engines may not understand the page structure.

The webpage becomes difficult to maintain and debug.

Using valid HTML ensures a consistent user experience on all devices and browsers.

5. Suggest corrections to make the DOM valid.

The corrected HTML document is shown below.

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Test Page</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Test Page</h1>
```

```
</header>
```

```
<main>
```

```
<section>
```

```
<div>
```

```
<p>Hello World</p>
```

```
</div>
```

```
</section>
```

```
</main>
```

```
<footer>

<p>© 2026 Test Page</p>

</footer>

</body>

</html>
```

Explanation

The corrected HTML includes:

- HTML5 doctype declaration.
- UTF-8 character encoding.
- Responsive viewport.
- Proper opening and closing tags.
- Semantic HTML5 elements.
- A valid DOM structure.

Benefits

- Better browser compatibility.
- Improved SEO.
- Easier maintenance.
- Better accessibility.
- Valid W3C-compliant HTML.
- Consistent rendering across all browsers.

Conclusion

A valid DOM structure is essential for creating reliable, maintainable, and standards-compliant web applications. Properly closing tags and using semantic HTML ensures that webpages render consistently across different browsers and devices.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.

GET and POST are two HTTP methods used to send data between the client (browser) and the server. Although both methods transfer data, they differ in the way they send information and their level of security.

GET Method

The GET method sends data by appending it to the URL as query parameters.

Example:

/login?user=admin&pass=1234

Characteristics of GET

- Data is visible in the URL.
- Data can be bookmarked.
- Data is stored in browser history.
- Suitable for searching and retrieving information.
- Less secure because anyone can view the URL.

POST Method

The POST method sends data inside the HTTP request body instead of the URL.

Example:

POST /login HTTP/1.1

user=admin

pass=1234

Characteristics of POST

- Data is hidden from the URL.
- Suitable for login forms and registration forms.
- Supports large amounts of data.
- More secure than GET.
- Cannot be bookmarked because data is not included in the URL.

Difference Table

Feature	GET	POST
Data Location	URL	Request Body
Visibility	Visible	Hidden
Security	Low	Moderate
Bookmark	Yes	No
Used For	Retrieve Data Submit Data	

Conclusion

GET is mainly used for retrieving information, whereas POST is used for securely sending user data to the server.

2. Analyze which method is more suitable for login and why.

For a login system, **POST** is the most suitable method.

Reasons

1. Username and password are not displayed in the browser address bar.
2. Login credentials are sent inside the request body.
3. Browser history does not store sensitive information.
4. Passwords are less likely to be exposed to other users.
5. POST supports HTTPS encryption.
6. Most secure authentication systems use POST together with HTTPS.

Example

Using GET

/login?user=admin&pass=1234

Anyone can see the password in the URL.

Using POST

POST /login

The username and password remain inside the request body and are not displayed in the URL.

Conclusion

POST provides better privacy and security. Therefore, it is the preferred method for login systems.

3. Identify security risks in using GET for authentication.

Using GET for login introduces several security risks.

1. Password Visible in URL

The username and password appear in the browser address bar.

Example:

/login?user=admin&pass=1234

Anyone nearby can see the credentials.

2. Browser History

The URL containing the password is stored in browser history.

3. Bookmarking

Users may accidentally bookmark the login URL containing sensitive information.

4. Server Logs

Web servers automatically record requested URLs in log files.

The password may be stored permanently in server logs.

5. Proxy Servers

Proxy servers may cache GET requests.

Sensitive information could be exposed to unauthorized users.

6. Sharing URLs

Users may accidentally copy and share URLs containing login credentials.

7. Security Attacks

Attackers may collect usernames and passwords from URLs using network monitoring tools.

Conclusion

Because of these security risks, GET should never be used for authentication or login forms.

4. Suggest best practices for secure login implementation.

To build a secure login system, the following best practices should be followed.

1. Use POST Method

Always send login credentials using the POST method.

2. Use HTTPS

HTTPS encrypts all communication between the browser and the server.

Example:

`https://www.example.com/login`

3. Validate User Input

Check username and password on both the client side and server side.

4. Store Passwords Securely

Passwords should never be stored as plain text.

Use secure hashing algorithms such as:

- bcrypt
- Argon2
- PBKDF2

5. Prevent SQL Injection

Use prepared statements or parameterized queries instead of directly inserting user input into SQL statements.

6. Implement Session Management

Create secure sessions after successful login.

Use secure cookies with:

- Secure
- HttpOnly
- SameSite

7. Enable CSRF Protection

Include CSRF tokens in login forms to prevent unauthorized requests.

8. Limit Login Attempts

Temporarily lock accounts after multiple failed login attempts to prevent brute-force attacks.

9. Display Generic Error Messages

Instead of showing:

Password is incorrect

Display:

Invalid username or password.

This prevents attackers from identifying valid usernames.

Example Secure Login Form

HTML

```
<form action="/login" method="POST">

<label>Username</label>
<input type="text" name="username" required>

<label>Password</label>
<input type="password" name="password" required>

<button type="submit">Login</button>

</form>
```

Conclusion

A secure login system should always use the POST method, HTTPS encryption, strong password hashing, server-side validation, secure session management, and protection against common web attacks. These practices help protect user credentials and ensure safe authentication.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.

Given HTML:

HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>

<body>
```

```
<h1>Welcome
```

```
<p>This is a paragraph
```

```

```

```
</body>
```

```
</html>
```

Missing Closing Tags

The following tags are not properly closed:

- `</h1>`
- `</p>`

The `<img>` element is a **void element**, so it does not require a closing tag.

Properly closing HTML tags improves browser compatibility and makes the code easier to read and maintain.

2. Analyze issues with the `<img>` tag.

The `<img>` tag is incomplete because it lacks important attributes.

Issues

- Missing **alt** attribute.
- No image width and height are specified.
- If the image file is missing, the browser cannot display alternative text.

Incorrect

HTML

```

```

Correct

HTML

```

```

Benefits of the alt Attribute

- Improves accessibility.
- Helps screen readers.
- Displays text if the image cannot load.
- Improves SEO.

3. Determine how browsers will handle these errors.

Modern browsers automatically try to fix HTML errors.

Possible browser behavior includes:

- Automatically closing missing tags.
- Rendering the page even with invalid HTML.
- Displaying inconsistent layouts.
- Applying CSS differently.
- Causing JavaScript DOM issues.

Different browsers may display the webpage differently because they use different rendering engines.

Although the webpage may appear correctly, invalid HTML can lead to maintenance and compatibility issues.

4. Suggest a corrected version of the code.

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Sample</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Welcome</h1>
```

```
</header>
```

```
<main>
```

```
<section>
```

```
<p>This is a paragraph.</p>
```

```

```

```
</section>
```

```
</main>
```

```
<footer>
```

```
<p>© 2026 Sample Website</p>
```

```
</footer>
```

```
</body>
```

```
</html>
```

Explanation

The corrected HTML includes:

- Proper opening and closing tags.
- HTML5 semantic elements.
- alt attribute for the image.
- Responsive meta tag.
- UTF-8 character encoding.
- Well-structured layout following W3C standards.

Conclusion

Writing valid HTML with proper closing tags and correctly configured images improves accessibility, browser compatibility, SEO, and webpage maintainability. Following HTML5 standards ensures that webpages display consistently across all modern browsers.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

Browser–Server communication is the process through which a web browser requests information from a web server and displays the requested webpage. Each step plays an important role in loading a website correctly.

Step 1: User enters a URL

The user types a website address such as:

`https://www.example.com`

The browser identifies the requested website and prepares to retrieve its content.

Step 2: DNS Lookup

A Domain Name System (DNS) server converts the domain name into an IP address.

Example:

`www.example.com` → `192.168.10.15`

The browser needs the IP address to locate the correct server on the Internet.

Step 3: HTTP Request is Sent

After obtaining the IP address, the browser sends an HTTP request to the web server.

Example:

`GET / HTTP/1.1`

Host: www.example.com

The request asks the server to provide the requested webpage.

Step 4: Server Processes the Request

The server receives the request and performs the necessary operations, such as:

- Checking the requested resource.
- Accessing the database.
- Executing server-side programs.
- Generating the webpage.

Step 5: Response is Returned

The server sends an HTTP response containing:

- Status code (200 OK)
- HTML page
- CSS files
- JavaScript files
- Images and other resources

Example:

HTTP/1.1 200 OK

Content-Type: text/html

Step 6: Browser Renders the Page

The browser interprets the HTML, CSS, and JavaScript files and displays the webpage to the user

.

Conclusion

Each step is essential for delivering a webpage correctly. If any step fails, the webpage may not load successfully.

2. Identify where delays can occur.

Although the communication process is fast, delays may occur at different stages.

1. DNS Lookup Delay

If the DNS server responds slowly, additional time is required to obtain the IP address.

2. Network Delay

Slow Internet connections increase the time required to transmit requests and responses.

3. Server Processing Delay

The server may require extra time to:

- Execute programs.
- Query databases.
- Process user requests.

4. Database Delay

Large databases or complex SQL queries may increase response time.

5. Large File Downloads

Large images, videos, CSS, or JavaScript files increase page loading time.

6. Browser Rendering Delay

Complex HTML pages and heavy JavaScript may slow down webpage rendering.

Effects of Delays

- Slow webpage loading.
- Poor user experience.
- Increased server load.
- Higher bounce rate.
- Reduced website performance

Conclusion

Optimizing each stage helps improve webpage speed and user satisfaction.

3. Analyze the role of DNS in the process.

DNS (Domain Name System) acts as the Internet's directory service. Humans remember domain names, while computers communicate using IP addresses.

Functions of DNS

- Converts domain names into IP addresses.
- Helps browsers locate web servers.
- Eliminates the need to remember numerical IP addresses.
- Makes web browsing easier and more user-friendly.

Example

Domain Name:

www.google.com

DNS converts it into an IP address such as:

142.250.183.206

The browser then contacts the server using this IP address.

Advantages of DNS

- Easy website access.
- Faster communication through DNS caching.
- Reliable server identification.
- Supports Internet scalability.

Without DNS

Users would have to enter numerical IP addresses for every website, making web browsing difficult and inconvenient.

Conclusion

DNS is a fundamental Internet service that enables browsers to locate web servers quickly and accurately.

4. Suggest optimizations to improve performance.

Several techniques can improve Browser–Server communication performance.

1. Enable Browser Caching

Frequently accessed files are stored locally, reducing repeated downloads.

2. Use DNS Caching

Caching DNS records reduces lookup time for repeated visits.

3. Compress Files

Enable Gzip or Brotli compression to reduce the size of HTML, CSS, and JavaScript files.

4. Optimize Images

Compress and resize images before uploading them to the website.

5. Minify CSS and JavaScript

Remove unnecessary spaces and comments to reduce file size.

6. Use a Content Delivery Network (CDN)

Store website content on multiple servers worldwide to reduce latency.

7. Optimize Database Queries

Use indexing and efficient SQL queries to reduce server processing time.

8. Use HTTPS with HTTP/2 or HTTP/3

Modern protocols improve speed and support multiple requests over a single connection.

9. Reduce Server Response Time

Use efficient server-side programming and upgraded hardware.

10. Monitor Website Performance

Use developer tools and performance monitoring software to identify bottlenecks and optimize the website.

Conclusion

Performance optimization improves webpage loading speed, reduces server load, enhances user experience, and ensures efficient browser–server communication.

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand